Islamic University of Gaza

Faculty of Engineering

Computer Engineering Department

Data Base Management System Course

DataBase Project

"Hospital Management System"

Prepared By

Mohammed Ayman Khader

120191065

Supervised by:

Dr. Belal Al_Farra

Eng. Ahmed al-kahlout

24th MAY, 26

Contents

List of Figures

List of Tables

Abstract

This project presents the design and implementation of a Hospital Management System database intended to efficiently manage hospital operations and medical records. The system was developed using fundamental Database Management System (DBMS) concepts and focuses on organizing and maintaining data related to patients, doctors, departments, appointments, prescriptions, and medicines.

The project follows a complete database design methodology starting from requirement analysis and Entity-Relationship Diagram (ERD) modeling, followed by the transformation into a relational schema and the refinement process using normalization techniques. The database structure was carefully designed to ensure data integrity, reduce redundancy, and maintain consistency through the use of primary keys, foreign keys, constraints, and relationship modeling.

The system supports several core functionalities such as patient registration, appointment scheduling, doctor specialization management, prescription handling, and medicine tracking. In addition, advanced database concepts including composite keys, multivalued attributes, self-referencing relationships, and integrity constraints were incorporated to create a realistic and efficient database design.

The final implementation was developed using SQL and demonstrates how database systems can be utilized to improve data organization, accessibility, and reliability in healthcare environments. This project highlights the importance of proper database design in building scalable and maintainable information systems.

Introduction

Modern healthcare institutions generate and manage large volumes of data on a daily basis, including patient records, doctor information, appointments, prescriptions, and medical treatments. Efficient management of this data is essential for ensuring high-quality healthcare services, reducing operational errors, and improving communication between different hospital departments.

The main problem addressed in this project is the inefficiency of traditional or manual hospital record management systems. Such systems often suffer from data redundancy, inconsistency, difficulty in retrieving information, and delays in handling patient-related processes such as appointments and prescriptions.

To overcome these challenges, this project proposes the design and implementation of a Hospital Management System database using the principles of Database Management Systems (DBMS). The system is designed to store and manage hospital data in a structured and organized manner. It includes key entities such as patients, doctors, departments, appointments, prescriptions, and medicines, all of which are interconnected through well-defined relationships.

The main objectives of this system are to design a structured and efficient database for managing hospital operations, to store and organize patient information including personal details and contact numbers, to manage doctor records and their specializations, and to handle hierarchical relationships among doctors. In addition, the system aims to manage hospital departments, schedule and track patient appointments, and store prescriptions along with related medicines. Furthermore, the system ensures data integrity, minimizes redundancy, and supports fast and reliable access to medical information for hospital staff.

Overall, this project demonstrates how a well-designed relational database can improve the efficiency, accuracy, and reliability of hospital data management systems, contributing to better healthcare service delivery.

Problem Description

Modern hospitals handle a large amount of sensitive and complex data, including patient information, doctor records, appointments, prescriptions, and medical departments. Managing this data using traditional or manual methods often leads to inefficiencies, data redundancy, inconsistency, and difficulties in retrieving accurate information in a timely manner.

The Hospital Management System database is designed to solve these issues by providing a structured and centralized system for storing and managing hospital data. The system ensures that all information is organized in relational tables with properly defined relationships, which improves data consistency and accessibility.

The system is intended to support different types of users, including administrators who manage the system and maintain data, doctors who handle patient treatment and prescriptions, and patients whose personal and medical information is stored in the system.

The main functional requirements of the system include managing patient records, doctor information, hospital departments, medical appointments, prescriptions, and medicines. The system also ensures proper handling of relationships between these entities, such as assigning doctors to departments, scheduling appointments between doctors and patients, and linking prescriptions with medications.

The primary objectives of the system are to design an efficient relational database structure, ensure data integrity and consistency, minimize redundancy, and provide fast and reliable access to hospital data. The system aims to improve the overall efficiency of hospital operations and support better decision-making through organized and accurate data management.

Conceptual Database Design (ER Diagram)

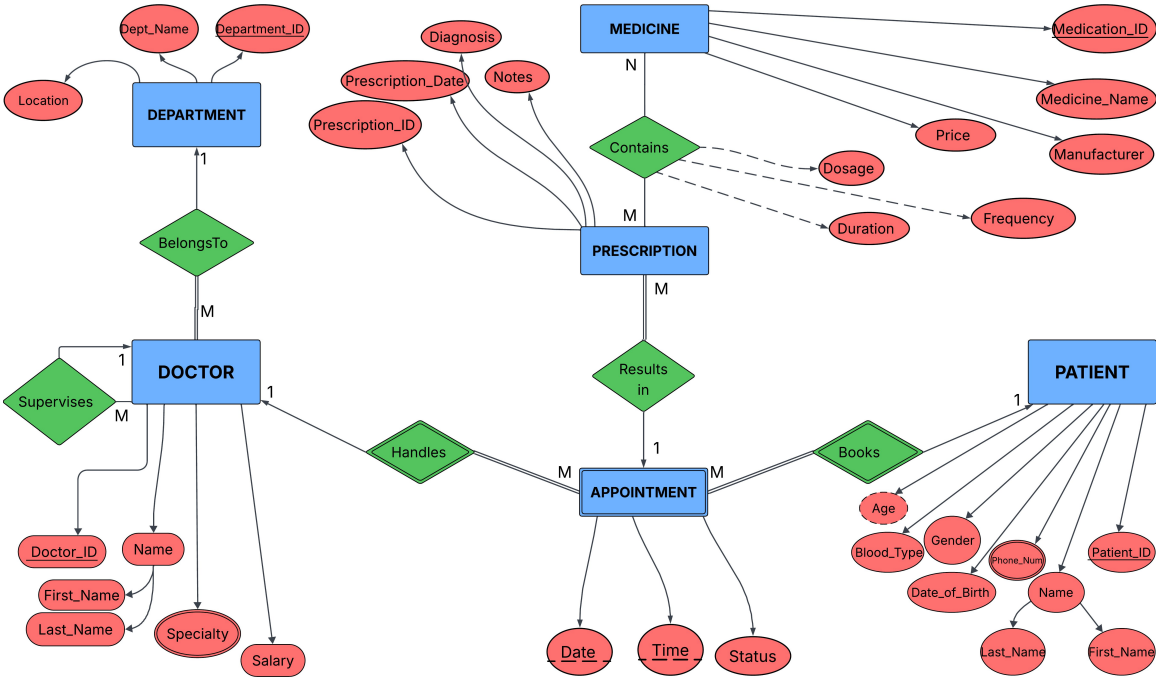


Figure 1: Entity Relationship Diagram of Hospital Management System

The Entity-Relationship (ER) diagram represents the conceptual design of the Hospital Management System database. It illustrates the main entities, their attributes, and the relationships between them, providing a clear overview of how data is organized within the system.

The system consists of several main entities including PATIENT, DOCTOR, DEPARTMENT, APPOINTMENT, PRESCRIPTION, and MEDICINE. Each entity contains a set of attributes that describe its properties. For example, the PATIENT entity includes personal information such as Patient_ID, First_Name, Last_Name, Date_of_Birth, Gender, Blood_Type, and Phone_Number. The DOCTOR entity includes details such as Doctor_ID, name, salary, and specialty.

The diagram also includes a weak entity, APPOINTMENT, represented using a double rectangle. This entity depends on both identifying entity sets, PATIENT and DOCTOR, and is uniquely identified through a composite key consisting of Doctor_ID, Patient_ID, together with the discriminator attributes Date and Time.

Although APPOINTMENT could have been designed as a strong entity—similar to PRESCRIPTION—by introducing a surrogate key such as *Appointment_ID*, this approach was intentionally avoided. Instead, the design deliberately omits a standalone identifier in order to emphasize and utilize the weak entity concept.

The two identifying entity sets are **PATIENT** and **DOCTOR**, and the two identifying relationships are **"books" (PATIENT–APPOINTMENT)** and **"assigned to" (DOCTOR–APPOINTMENT)**.

Relationships between entities are clearly defined. A doctor belongs to a department through a many-to-one relationship, while each department can have multiple doctors. Doctors may also supervise other doctors using a recursive relationship.

The PRESCRIPTION entity represents medical prescriptions that result from a specific appointment. It is directly associated with the APPOINTMENT entity through the relationship "Results in", meaning that each prescription is generated as a consequence of a completed medical appointment. The entity stores essential clinical information such as diagnosis, prescription date, and additional notes, ensuring that prescriptions are always linked to a valid and recorded patient–doctor consultation.

A many-to-many relationship exists between PRESCRIPTION and MEDICINE, which is resolved through the associative entity PRESCRIPTION_MEDICINE. This associative entity stores additional information about each prescribed medicine, including dosage, frequency, and duration.

The participation of the PRESCRIPTION entity in this relationship depends on the business rules of the system. Initially, it was assumed that every prescription must contain at least one medicine, which would imply Total Participation of PRESCRIPTION in the relationship. However, in the proposed Hospital Management System, a prescription may consist solely of non-medication instructions such as rest recommendations, dietary guidance, follow-up advice, or other medical notes without requiring any prescribed medicine.

As a result, a prescription can exist independently of the MEDICINE entity. Therefore, participation of PRESCRIPTION in the Contains relationship is considered Partial Participation rather than Total Participation.

Participation Summary:

PRESCRIPTION → CONTAINS → MEDICINE = Partial Participation

Overall, the ER diagram uses standard notation to represent entities, attributes, relationships, and constraints. Solid lines represent identifying relationships, while dashed lines indicate derived or non-identifying attributes. The diagram ensures data consistency, minimizes redundancy, and provides a clear structure for implementing the relational database.

The SUPERVISES relationship is a recursive (self-referencing) relationship on the DOCTOR entity, where a doctor can supervise other doctors within the same entity set. This type of relationship is commonly used to model hierarchical or organizational structures, such as hospital staffing systems, residency programs, or clinical training environments, where less experienced doctors are guided and monitored by senior doctors.

In this system, the role of supervision depends on the professional level and assignment of each doctor. Some doctors, particularly junior doctors or trainees (such as interns or residents), are assigned to a supervising doctor to ensure proper clinical guidance and oversight. On the other hand, senior or fully qualified doctors may supervise others, while some may not participate in the supervisory role at all.

Therefore, participation in the SUPERVISES relationship is optional on both sides.

Cardinality Summary:

- A doctor may supervise zero or many doctors.
- A doctor may be supervised by zero or one doctor.

Thus, the relationship is characterized as:
DOCTOR → SUPERVISES → DOCTOR = Partial Participation (on both roles)

This recursive structure ensures flexibility in representing different levels of medical hierarchy without forcing every doctor to be either a supervisor or a subordinate.

Relational Schema Design

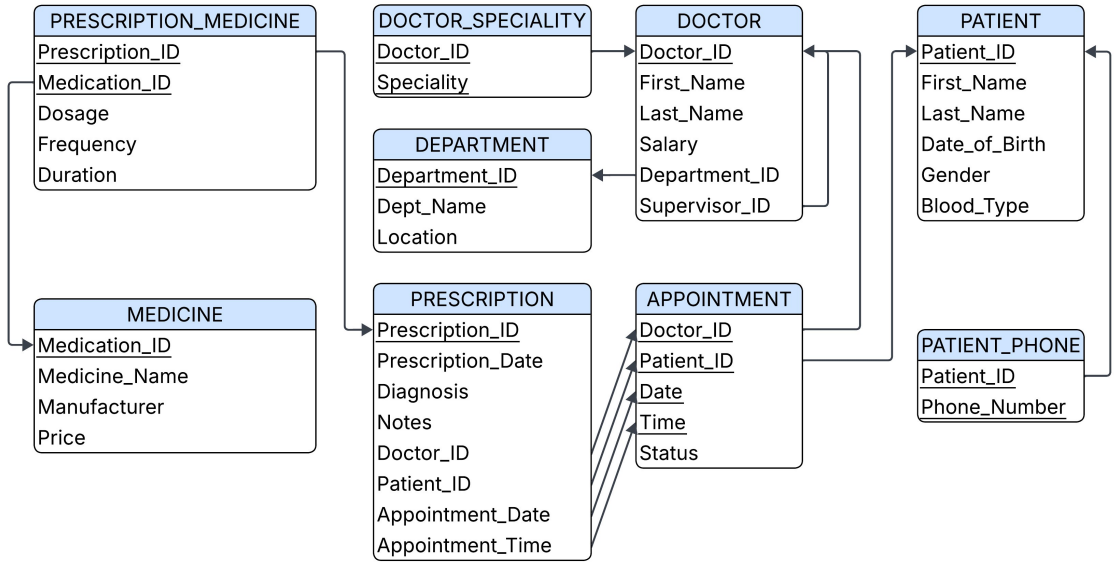


Figure 2: Relational Schema Diagram of the Proposed System

The relational schema was derived from the conceptual ER model using standard transformation rules that map entities, relationships, and attributes into relational tables. The main objective of this transformation is to ensure a consistent, efficient, and well-structured database design that eliminates redundancy and avoids data anomalies.

撒 ER-to-Relational Mapping Rules

During the conversion process, the following rules were applied:

- Each strong entity is transformed into a separate relation with its primary key preserved.
- Each multi-valued attribute is converted into a separate relation to ensure atomicity.
- Each one-to-many relationship is implemented by adding a foreign key on the “many” side.
- Each many-to-many relationship is transformed into a new associative relation containing the primary keys of the participating entities.
- The weak entity (APPOINTMENT) is mapped into a relation that includes the primary keys of its identifying entities (DOCTOR and PATIENT) along with its discriminator attributes (Date and Time).
- The recursive relationship (SUPERVISES) is implemented using a self-referencing foreign key (Supervisor_ID) within the DOCTOR relation.

These rules ensure that the relational model accurately reflects the original ER structure while maintaining data integrity.

Normalization Process (UNF → 3NF) for the Doctor–Appointment Module

This section demonstrates the step-by-step normalization process applied to the Doctor–Appointment module in the proposed medical system. The objective is to transform an unnormalized relation into a well-structured set of relations that satisfy First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF), while eliminating redundancy and ensuring data integrity.

In this example:

- **Patient_Name is a composite attribute: *(First_Name, Last_Name)*.**
- **Doctor_Name is a composite attribute: *(Doctor_First_Name, Doctor_Last_Name)*.**
- **Doctor_Specialty is a multivalued attribute (a doctor may have multiple specialties).**
- Each doctor belongs to a department identified by **Department_ID** and **Department_Name**.

1. Unnormalized Form (UNF)

In the UNF stage, all attributes, including composite and multivalued attributes, are stored within a single relation without normalization constraints.

PatientID	PatientName	DoctorID	DoctorName	DoctorSpecialty	DepartmentID	DepartmentName	AppointmentDate	AppointmentTime
1	Ali, Hassan	10	Dr. Ahmed Saleh	Cardiology, Heart Surgery	1	Cardiology	2026-05-20	10:00
1	Ali, Hassan	11	Dr. Omar Yousef	Neurology, Brain Disorders	2	Neurology	2026-05-21	12:00

Table 1: APPOINTMENT_UNF

Issues Identified

- Composite attribute: Patient_Name is not atomic.
- Composite attribute: Doctor_Name is not atomic.
- Multivalued attribute: Doctor_Specialty contains multiple values.
- Repetition of department information for doctors.
- High redundancy in patient, doctor, and department data.
- Update anomalies and inconsistency risks.
- Poor overall data structure.

2. First Normal Form (1NF)

A relation is in 1NF if:

- All attributes contain atomic values.
- No composite attributes exist.
- No multivalued attributes exist.

Step 1: Decompose Composite Attributes

Patient_Name → (First_Name, Last_Name)

Doctor_Name → (Doctor_First_Name, Doctor_Last_Name)

Step 2: Remove Multivalued Attributes

Doctor_Specialty is separated into a new relation.

Result

All attributes are now atomic, and multivalued attributes have been removed.

PatientID	FirstName	LastName	DoctorID	FirstName	LastName	DepartmentID	DepartmentName	AppointmentDate	AppointmentTime
1	Ali	Hassan	10	Ahmed	Saleh	1	Cardiology	2026-05-20	10:00
1	Ali	Hassan	11	Omar	Yousef	2	Neurology	2026-05-21	12:00

Table 2: Resulting 1NF Relations

Doctor_ID	**Specialty**
Dr. Ahmed	Cardiology
Dr. Ahmed	Heart Surgery
Dr. Samer	Neurology
Dr. Samer	Brain Disorders

Table 3: Resulting 1NF Relations

3. Second Normal Form (2NF)

A relation is in 2NF if:

- It is in 1NF
- There is no partial dependency (non-key attributes must depend on the whole primary key)

Assume the composite key in APPOINTMENT:
(Patient_ID, Doctor_ID, Appointment_Date, Appointment_Time)

Functional Dependencies:

- First_Name, Last_Name → depend only on Patient_ID
- Doctor_Name → depends only on Doctor_ID
- Department_ID and Department_Name depend on Doctor_ID.
- Appointment details → depend on full key

Patient_ID	**First_Name**	**Last_Name**
1	Ali	Hassan

Table 4: Resulting 2NF Relations

Doctor_ID	**FirstName**	**LastName**	**DepartmentID**	**DepartmentName**
10	Ahmed	Saleh	1	Cardiology
11	Omar	Yousef	2	Neurology

Table 4: Resulting 2NF Relations

Doctor_ID	**Specialty**
Dr. Ahmed	Cardiology
Dr. Ahmed	Heart Surgery
Dr. Samer	Neurology
Dr. Samer	Brain Disorders

Table 6: Resulting 2NF Relations

Patient_ID	**Doctor_ID**	**Appointment_Date**	**Appointment_Time**
1	10	2026-05-20	10:00
1	11	2026-05-21	12:00

Table 7: Resulting 2NF Relations

Result:

- Removal of partial dependencies
- Each table represents a single entity
- Significant reduction in redundancy.

4. Third Normal Form (3NF):

A relation is in 3NF if:

- It is in 2NF
- There are no transitive dependencies

Transitive Dependency Identified

Within the DOCTOR relation:

Doctor_ID→Department_ID
Department_ID→Department_Name

Therefore:

Doctor_ID→Department_Name

This is a transitive dependency and must be removed.

Final 3NF Schema

Patient_ID	**First_Name**	**Last_Name**
1	Ali	Hassan

Table 8: Resulting 3NF Relations

DepartmentID	DepartmentName
1	Cardiology
2	Neurology

Table 9: DEPARTMENT

Doctor_ID	FirstName	LastName	DepartmentID
10	Ahmed	Saleh	1
11	Omar	Yousef	2

Table 10: DOCTOR

Doctor_ID	Specialty
Dr. Ahmed	Cardiology
Dr. Ahmed	Heart Surgery
Dr. Samer	Neurology
Dr. Samer	Brain Disorders

Table 11: DOCTOR_SPECIALTY

Patient_ID	Doctor_ID	Appointment_Date	Appointment_Time
1	10	2026-05-20	10:00
1	11	2026-05-21	12:00

Table 12: APPOINTMENT

Conclusion

After applying normalization from UNF to 3NF, the original unstructured relation has been decomposed into multiple well-structured relations. This process eliminated composite and multivalued attributes, removed partial dependencies, and ensured that no transitive dependencies exist. As a result, the final database design achieves high data integrity, reduced redundancy, and improved consistency across the system.

SQL Implementation (Physical Design)

1. DDL: CREATE TABLE Statements

This stage represents the physical implementation of the database structure by translating the relational schema into actual database tables using SQL Data Definition Language (DDL).

The database consists of several interconnected tables that represent the main entities of the hospital management system, including patients, doctors, appointments, prescriptions, and medicines.

The PATIENT table stores patient personal information such as first name, last name, date of birth, gender, and blood type, with constraints ensuring valid and consistent data values.

To support multi-valued attributes, a separate PATIENT_PHONE table is implemented, allowing each patient to have multiple phone numbers while maintaining normalization and referential integrity.

The DEPARTMENT table represents hospital departments and includes department details such as department name and location.

The DOCTOR table stores doctor information including name, salary, and relationships to departments and supervisors. Self-referencing is used to represent hierarchical relationships between doctors.

Doctor specialties are managed through the DOCTOR_SPECIALITY table, which handles the many-to-many relationship between doctors and their medical specializations.

The APPOINTMENT table represents scheduled interactions between patients and doctors, including date, time, and status (Scheduled, Completed, Cancelled, Expired). It ensures proper linkage between patient and doctor entities through foreign keys.

The MEDICINE table contains details of available medicines, including name, manufacturer, and price, with constraints to ensure valid pricing values.

The PRESCRIPTION table links prescriptions to specific appointments, ensuring that each prescription is associated with a valid doctor–patient interaction. This table enforces referential integrity through a composite foreign key referencing the APPOINTMENT table.

Finally, the PRESCRIPTION_MEDICINE table manages the many-to-many relationship between prescriptions and medicines, including additional attributes such as dosage, frequency, and duration.

All database views that support data retrieval and reporting operations have been implemented separately in the views file, which includes:

- Patient age calculation view
- Completed appointments view
- Patient medical history view
- Doctor workload statistics view
- Prescription summary views

These views are designed to simplify complex queries, improve readability, and provide structured access to frequently required information.

2. DML: Data Manipulation Statements

This stage involves populating the database with realistic sample data to simulate real hospital operations and to test the functionality of the system.

The database was populated with meaningful datasets representing different entities in the system. This includes:

- 10 patient records, each containing patient ID, name, date of birth, gender, and blood type. Additionally, some patients were assigned multiple phone numbers to represent multi-valued attributes in a real-world scenario.
- 5 hospital departments to organize medical specializations.
- 10 doctors, each associated with a department, salary, and supervisor relationship where applicable.
- 15 doctor specialties to represent the relationship between doctors and their medical specializations.
- 20 appointment records linking patients and doctors, each containing appointment date, time, and status (Scheduled, Completed, Cancelled, Expired). Each patient is associated with two appointments to ensure realistic workload distribution.
- 20 medicines with details including medicine ID, name, manufacturer, and price.
- 20 prescription records representing issued medical prescriptions.
- 20 entries in the Prescription_Medicine relationship table to manage the many-to-many relationship between prescriptions and medicines.

This dataset was carefully designed to reflect real-world hospital operations and to test relationships, constraints, and business rules implemented in the system.

All data insertion statements are included in a separate file:**(DML.sql)**

3. Database Triggers and Business Rules

To ensure data integrity and enforce advanced business rules beyond standard constraints, several database triggers and functions were implemented.

These triggers handle important system-level validations, including:

- Preventing the insertion of a prescription unless the related appointment status is marked as Completed.
- Preventing the creation of appointments with past dates to ensure valid scheduling.
- Limiting the number of patients assigned to a single doctor as part of workload management rules.
- Preventing duplicate appointments for the same doctor at the same date and time.
- Preventing deletion of a doctor who still has active appointments in the system.

These triggers ensure that the database maintains consistency, enforces real-world constraints, and prevents invalid or conflicting operations.

All trigger definitions and functions are included in a separate file:

(triggers&functions.sql).

4. SQL Queries

To demonstrate the functionality and effectiveness of the implemented database system, five SQL queries were developed and tested using the populated hospital database. These queries were designed to cover different SQL concepts and operations, including joins, recursive queries, window functions, aggregation, and advanced analytical operations.

The implemented queries include:

- A JOIN query to display the appointment history of patients along with their associated doctors.
- A recursive query using Recursive Common Table Expressions (CTE) to retrieve the transitive closure of doctor supervision relationships.
- A window function query that calculates the cumulative number of appointments for each doctor over time using ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

- An aggregation query that identifies the most prescribed medicines based on prescription frequency.
- An advanced aggregation query that calculates statistical information about medicine prices per manufacturer, including average, maximum, and minimum prices.

These queries were implemented to validate database relationships, demonstrate advanced SQL capabilities, and support realistic hospital data analysis and reporting operations.

For better organization, all SQL query statements are provided separately in the following file: (Queries.sql)

The execution results and output screenshots of these queries are presented in the Results section of this report.

Results: **Table Creation Screenshot**

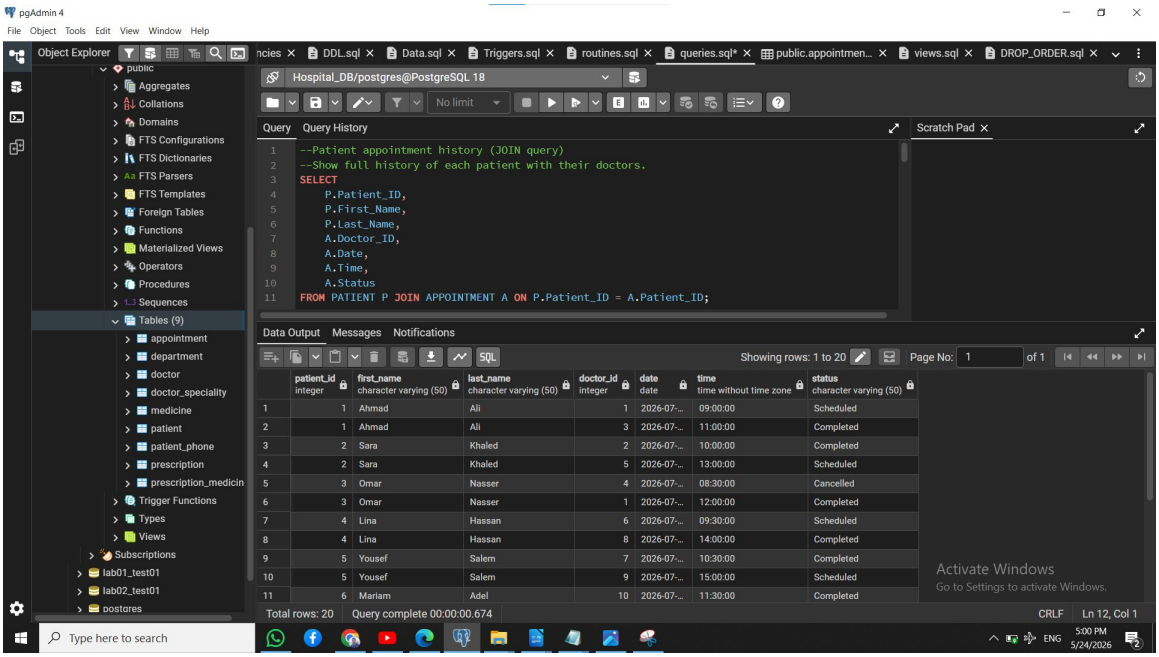
Query | Query History

```sql
CREATE TABLE PATIENT (
    Patient_ID INT PRIMARY KEY,
    First_Name VARCHAR(50) NOT NULL,
    Last_Name VARCHAR(50) NOT NULL,
    Date_of_Birth DATE NOT NULL,
    Gender VARCHAR(10) CHECK (Gender IN ('Male','Female')),
    Blood_Type VARCHAR(5) CHECK (
        Blood_Type IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')
    )
);


CREATE TABLE PATIENT_PHONE (
    Patient_ID     INT NOT NULL,
    Phone_Number   VARCHAR(15) NOT NULL,
    PRIMARY KEY (Patient_ID, Phone_Number),
    FOREIGN KEY (Patient_ID)
        REFERENCES PATIENT(Patient_ID)
        ON DELETE CASCADE
);

CREATE TABLE DEPARTMENT (
    Department_ID INT NOT NULL,
    Dept_Name VARCHAR(100) NOT NULL,
    Location VARCHAR(100),

    CONSTRAINT PK_DEPARTMENT
        PRIMARY KEY (Department_ID)
);

CREATE TABLE DOCTOR (
```

Data Insertion Screenshots

```sql
INSERT INTO PATIENT
VALUES
(1, 'Ahmad', 'Ali', '2001-03-15', 'Male', 'O+'),
(2, 'Sara', 'Khaled', '1999-07-22', 'Female', 'A+'),
(3, 'Omar', 'Nasser', '1988-11-02', 'Male', 'B+'),
(4, 'Lina', 'Hassan', '2005-01-30', 'Female', 'AB+'),
(5, 'Yousef', 'Salem', '1995-09-18', 'Male', 'O-'),
(6, 'Mariam', 'Adel', '2000-06-12', 'Female', 'A-'),
(7, 'Kareem', 'Fadi', '1993-04-27', 'Male', 'B-'),
(8, 'Dana', 'Ibrahim', '2002-12-09', 'Female', 'O+'),
(9, 'Tariq', 'Mahmoud', '1985-08-14', 'Male', 'AB-'),
(10, 'Nour', 'Samir', '1998-10-05', 'Female', 'A+');


INSERT INTO PATIENT_PHONE
VALUES
(1, '0599123456'),
(1, '0568123456'),

(2, '0599234567'),
(2, '0568234567'),
(2, '0599345678'),

(3, '0599456789'),
(3, '0568456789'),

(4, '0599567890'),
(4, '0568567890'),
(4, '0599678901');

INSERT INTO DEPARTMENT
```

Query Execution Screenshot & Output Results Screenshot



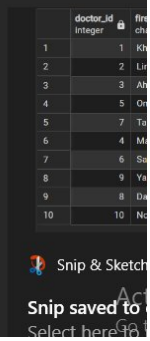
Query | Query History

```sql
-- Recursive Query (Transitive Closure of Doctor Supervision)
-- Goal:
-- Retrieve all doctors directly and indirectly supervised
-- by Doctor_ID = 1.

WITH RECURSIVE Doctor_Hierarchy AS (

    -- Base case: start from Doctor 1
    SELECT
        Doctor_ID,
        First_Name,
        Last_Name,
        Supervisor_ID
    FROM DOCTOR
    WHERE Doctor_ID = 1

    UNION ALL

    -- Recursive step: find doctors supervised by current doctors
    SELECT
        D.Doctor_ID,
        D.First_Name,
        D.Last_Name,
        D.Supervisor_ID
    FROM DOCTOR D
    INNER JOIN Doctor_Hierarchy DH
        ON D.Supervisor_ID = DH.Doctor_ID
)

SELECT *
FROM Doctor_Hierarchy;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No:

	doctor_id integer	first_name character varying (50)	last_name character varying (50)	supervisor_id integer
1	1	Khaled	Nasser	[null]
2	2	Lina	Hassan	1
3	3	Ahmad	Saleh	1
4	5	Omar	Yousef	1
5	7	Tariq	Fadi	1
6	4	Mariam	Adel	2
7	6	Sara	Mahmoud	3
8	9	Yazan	Ibrahim	2
9	8	Dana	Khalil	4
10	10	Nour	Samer	6



Query | Query History

```sql
--Display appointments for each doctor with a cumulative count of appointments over time
SELECT
    Doctor_ID,
    Date,
    Time,
    Status,

    COUNT(*) OVER (
        PARTITION BY Doctor_ID
        ORDER BY Date, Time
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS Running_Total_Appointments

FROM APPOINTMENT
ORDER BY Doctor_ID, Date, Time;
```

Data Output | Messages | Notifications

Showing rows: 1 to 20

	doctor_id integer	date date	time time without time zone	status character varying (50)	running_total_appointments bigint
1	1	2026-07-...	09:00:00	Scheduled	1
2	1	2026-07-...	12:00:00	Completed	2
3	2	2026-07-...	10:00:00	Completed	1
4	2	2026-07-...	16:00:00	Scheduled	2
5	3	2026-07-...	11:00:00	Completed	1
6	3	2026-07-...	09:15:00	Cancelled	2
7	4	2026-07-...	08:30:00	Cancelled	1
8	4	2026-07-...	10:45:00	Scheduled	2
9	5	2026-07-	13:00:00	Scheduled	1

Query | Query History

```sql
--Display medicines ordered by the number of times they were prescribed.
SELECT
    M.Medicine_Name,
    COUNT(PM.Prescription_ID) AS Times_Prescribed
FROM MEDICINE M JOIN PRESCRIPTION_MEDICINE PM ON M.Medication_ID = PM.Medication_ID
GROUP BY M.Medicine_Name
ORDER BY Times_Prescribed DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 15

	medicine_name character varying (150)	times_prescribed bigint
1	Cetirizine	3
2	Diclofenac	2
3	Paracetamol	2
4	Hydrocortisone	2
5	Losartan	1
6	Omeprazole	1
7	Clopidogrel	1
8	Insulin	1
9	Ibuprofen	1
10	Amoxicillin	1
11	Lisinopril	1
12	Atorvastatin	1
13	Metformin	1
14	Aspirin	1
15	Salbutamol	1

l_DB > Schemas > public > Tables > doctor 378

Query Query History

```sql
--Average Medicine Price per Manufacturer
SELECT
    Manufacturer,
    COUNT(*) AS Total_Medicines,
    AVG(Price) AS Average_Price,
    MAX(Price) AS Highest_Price,
    MIN(Price) AS Lowest_Price
FROM MEDICINE
GROUP BY Manufacturer
HAVING COUNT(*) >= 2;
```

Data Output Messages Notifications

Showing rows: 1 to 6

	manufacturer character varying (150)	total_medicines bigint	average_price numeric	highest_price numeric	lowest_price numeric
1	Pfizer	4	12.2250000000000000	18.40	5.50
2	Teva	2	11.8000000000000000	14.30	9.30
3	Sanofi	2	18.1500000000000000	19.50	16.80
4	Novartis	2	12.2250000000000000	15.75	8.70
5	Merck	2	13.2000000000000000	13.50	12.90
6	Bayer	2	5.7250000000000000	7.25	4.20

Conclusions and Recommendations

Conclusions

This project successfully achieved the design and implementation of a Hospital Management System database using a structured relational approach. The system was developed starting from the conceptual design (ER Diagram), followed by relational schema mapping, normalization (UNF to 3NF), and finally physical implementation using SQL.

The normalization process helped eliminate data redundancy and ensured data integrity across all relations. The final database design provides a well-structured and efficient system for managing patients, doctors, appointments, prescriptions, medicines, and related hospital operations.

In addition, the implementation of advanced SQL features such as JOIN operations, recursive queries, window functions, and aggregation queries demonstrates the system's ability to support complex data retrieval and analysis requirements in a real-world healthcare environment.

Recommendations

- Future enhancements can include implementing a graphical user interface (GUI) to improve user interaction with the system.
- Additional security mechanisms such as role-based access control (RBAC) should be applied to protect sensitive medical data.
- The system can be extended to support real-time analytics and reporting dashboards for hospital management.
- Further optimization of queries and indexing strategies can improve performance for large-scale datasets.
- Integration with external systems such as pharmacy or laboratory systems can enhance overall hospital automation.

References

The following tools and technologies were used in the design and implementation of this project:

- **Lucidchart – Used for designing the Entity-Relationship (ER) Diagram and conceptual modeling of the system.**
- **draw.io (diagrams.net) – Used for creating and refining database diagrams and visual representations of the system structure.**
- **pgAdmin – Used as the primary database management tool for implementing, testing, and executing SQL scripts, including DDL, DML, and query operations on a PostgreSQL database system.**
- **PostgreSQL Documentation – Referenced for SQL syntax, constraints, and advanced features such as recursive queries and window functions.**
- **Course Lecture Materials – Used as a guideline for database design principles, normalization rules, and relational schema development.**